Strings

23L. String Algorithms (12 problems)

Pattern recognition: Text manipulation, pattern matching, frequency counting, encoding.

23.151 Valid Anagram

JAVASCRIPT Copy

```
function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const count = Array(26).fill(0);
  for (let i = 0; i < s.length; i++) {
    count[s.charCodeAt(i) - 97]++;
    count[t.charCodeAt(i) - 97]--;
  }
  return count.every(c => c === 0);
}
```

23.152 Group Anagrams

Approach: Group strings by a canonical key made from their sorted characters, using a hash map from key to the list of anagrams.

JAVASCRIPT Copy

```
  for (const s of strs) {
    const key = [...s].sort().join('');
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(s);
  }
  return [...map.values()];
}
```

Sort each word as key, group by sorted key in Map.

23.153 Longest Common Prefix

Approach: Start with the first string as the candidate prefix and repeatedly trim it until it prefixes every other string.

JAVASCRIPT Copy

```
function longestCommonPrefix(strs) {
  if (strs.length === 0) return '';
  let prefix = strs[0];
  for (let i = 1; i < strs.length; i++) {
    while (!strs[i].startsWith(prefix)) {
      prefix = prefix.slice(0, -1);
      if (prefix === '') return '';
    }
  }
  return prefix;
}
```

Compare character by character across all strings. Stop when any differs.

23.154 String to Integer (atoi)

Approach: Skip leading spaces, read an optional sign, accumulate digits until a non-digit, then clamp the result to the signed 32-bit integer range.

JAVASCRIPT Copy

```
function myAtoi(s) {
  const INT_MAX = 2147483647, INT_MIN = -2147483648;
  let i = 0, n = s.length;
  while (i < n && s[i] === ' ') i++;
  let sign = 1;
  if (i < n && (s[i] === '+' || s[i] === '-')) {
    if (s[i] === '-') sign = -1;
    i++;
  }
  let num = 0;
  while (i < n && s[i] ≥ '0' && s[i] ≤ '9') {
    num = num * 10 + (s.charCodeAt(i) - 48);
    i++;
  }
  num *= sign;
  if (num < INT_MIN) return INT_MIN;
  if (num > INT_MAX) return INT_MAX;
  return num;
}
```

Trim whitespace, handle sign, parse digits, clamp to INT_MIN/INT_MAX.

23.155 KMP Pattern Matching

JAVASCRIPT Copy

```
function kmp(text, pattern) {
  const lps = buildLPS(pattern);
  let i = 0, j = 0;
  while (i < text.length) {
    if (text[i] === pattern[j]) { i++; j++; }
    if (j === pattern.length) return i - j;
    else if (i < text.length && text[i] =≠ pattern[j]) {
      if (j > 0) j = lps[j - 1];
```

```
  return -1;
}

function buildLPS(pattern) {
  const lps = Array(pattern.length).fill(0);
  let len = 0, i = 1;
  while (i < pattern.length) {
    if (pattern[i] === pattern[len]) { lps[i++] = ++len; }
    else if (len > 0) len = lps[len - 1];
    else lps[i++] = 0;
  }
  return lps;
}
```

23.156 Reverse Words in String

Approach: Trim the string, split on runs of whitespace, reverse the resulting word array, and join with single spaces.

JAVASCRIPT Copy

```
function reverseWords(s) {
  return s.trim().split(/\s+/).reverse().join(' ');
}
```

Split by whitespace, filter empty, reverse, join.

23.157 Zigzag Conversion

Approach: Walk the string appending each character to its current row, bouncing the row direction at the top and bottom, then concatenate all rows.

```javascript
function convert(s, numRows) {
  if (numRows === 1) return s;
  const rows = Array.from({ length: numRows }, () => '');
  let cur = 0, dir = -1;
  for (const ch of s) {
    rows[cur] += ch;
    if (cur === 0 || cur === numRows - 1) dir = -dir;
    cur += dir;
  }
  return rows.join('');
}
```

Simulate rows, change direction at top and bottom.

23.158 Count and Say

Approach: Start from '1' and iteratively build the next term by run-length encoding the current term: count consecutive equal digits and append count followed by the digit.

JAVASCRIPT Copy

```javascript
function countAndSay(n) {
  let s = '1';
  for (let k = 1; k < n; k++) {
    let next = '', i = 0;
    while (i < s.length) {
      let count = 1;
      while (i + 1 < s.length && s[i] === s[i + 1]) { count++; i++; }
      next += count + s[i];
      i++;
    }
    s = next;
  }
  return s;
}
```

23.159 Longest Substring with At Most K Distinct Characters

Approach: Slide a window while tracking character counts in a map, shrinking from the left whenever the distinct count exceeds k, and record the largest window size.

JAVASCRIPT Copy

```
function lengthOfLongestSubstringKDistinct(s, k) {
  if (k === 0) return 0;
  const count = new Map();
  let left = 0, best = 0;
  for (let right = 0; right < s.length; right++) {
    count.set(s[right], (count.get(s[right]) || 0) + 1);
    while (count.size > k) {
      const c = s[left];
      count.set(c, count.get(c) - 1);
      if (count.get(c) === 0) count.delete(c);
      left++;
    }
    best = Math.max(best, right - left + 1);
  }
  return best;
}
```

Sliding window + Map tracking character counts.

23.160 String Compression

Approach: Use read and write pointers to compress in place: for each run write the character and, if its length exceeds one, the run-length digits, returning the new length.

JAVASCRIPT Copy

```
  while (read < chars.length) {
    const ch = chars[read];
    let count = 0;
    while (read < chars.length && chars[read] === ch) { read++; count++; }
    chars[write++] = ch;
    if (count > 1) {
      for (const d of String(count)) chars[write++] = d;
    }
  }
  chars.length = write;
  return write;
}
```

Run-length encoding. "aabcccccaaa" → "a2b1c5a3".

23.161 Multiply Strings

Approach: Simulate grade-school multiplication into a digit array where the product of positions i and j lands at indices i+j and i+j+1, then strip leading zeros.

JAVASCRIPT Copy

```
function multiply(num1, num2) {
  if (num1 === '0' || num2 === '0') return '0';
  const m = num1.length, n = num2.length;
  const res = new Array(m + n).fill(0);
  for (let i = m - 1; i >= 0; i--) {
    for (let j = n - 1; j >= 0; j--) {
      const mul = (num1.charCodeAt(i) - 48) * (num2.charCodeAt(j) - 48);
      const sum = mul + res[i + j + 1];
      res[i + j + 1] = sum % 10;
      res[i + j] += Math.floor(sum / 10);
    }
  }
  return res.join('').replace(/^0+/, '');
}
```

23.162 Rabin-Karp (Rolling Hash)

Approach: Precompute the pattern hash and a rolling hash of each text window, sliding the window in O(1) per step and confirming candidate matches with a direct comparison to avoid false positives.

JAVASCRIPT Copy

```
function rabinKarp(text, pattern) {
  const n = text.length, m = pattern.length;
  if (m === 0) return 0;
  if (m > n) return -1;
  const base = 256, mod = 1000000007;
  let patHash = 0, winHash = 0, high = 1;
  for (let i = 0; i < m - 1; i++) high = (high * base) % mod;
  for (let i = 0; i < m; i++) {
    patHash = (patHash * base + pattern.charCodeAt(i)) % mod;
    winHash = (winHash * base + text.charCodeAt(i)) % mod;
  }
  for (let i = 0; i ≤ n - m; i++) {
    if (patHash === winHash && text.slice(i, i + m) === pattern) return i;
    if (i < n - m) {
      winHash = ((winHash - text.charCodeAt(i) * high % mod + mod) * base
    }
  }
  return -1;
}
```

Hash-based string matching. O(n+m) average case. Roll the hash window instead of recomputing.

QUICK CHECK — no marks, just for you.

1. The lesson's Valid Anagram solution buckets characters with charCodeAt(i) - 97. What does this snippet print?

```javascript
function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const count = Array(26).fill(0);
  for (let i = 0; i < s.length; i++) {
    count[s.charCodeAt(i) - 97]++;
    count[t.charCodeAt(i) - 97]--;
  }
  return count.every(c => c === 0);
}

console.log(isAnagram('listen', 'silent'));
console.log(isAnagram('Listen', 'silent'));
```

- ○ true
true
- ○ true
false
- ○ false
false
- ○ RangeError

Check answer

2. This is the buildLPS helper from the lesson's KMP implementation. What does it print?

JAVASCRIPT Copy

```javascript
function buildLPS(pattern) {
  const lps = Array(pattern.length).fill(0);
  let len = 0, i = 1;
  while (i < pattern.length) {
    if (pattern[i] === pattern[len]) { lps[i++] = ++len; }
    else if (len > 0) len = lps[len - 1];
    else lps[i++] = 0;
  }
  return lps;
```

- `0,1,0,1,2`
- `0,1,2,3,4`
- `0,0,0,1,2`
- `0,1,0,1,1`

Check answer

3. According to the lesson, what lets Rabin-Karp reach O(n+m) average-case string matching?

- It sorts the characters of the pattern first so each window comparison becomes a single O(1) key lookup.
- It updates the window's hash incrementally as the window slides, instead of recomputing the hash from scratch each time.
- It compares window lengths first and skips any window whose length cannot match the pattern.
- It precomputes an LPS table so the text pointer never has to backtrack after a mismatch.

Check answer

Frontend System Desi...

LEARN MORE

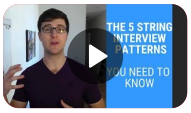
ARTICLES & DOCS

String data structure (GfG)
geeksforgeeks.org

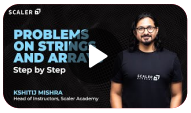
VIDEOS



Strings for coding interviews — full course
YouTube



The 5 string interview patterns
YouTube



String & array problems step by step
YouTube

Hand-picked references for this lesson.